

← Go Back

Hardware

Nano RP2040 Connect

Tutorials

- Nano RP2040 Connect User Manual
- Web Server AP Mode with Arduino Nano RP2040 Connect
- BLE Device to Device with Nano RP2040 Connect
- Nano RP2040 Connect Chromebook Setup
- Nano RP2040 Datalogger with MicroPython
- Using the IMU Machine Learning Core Features
- Accessing IMU Data on Nano RP2040 Connect
- Setting up Nano RP2040 Connect with Arduino Cloud
- Reading Microphone Data on Nano RP2040 Connect
- MicroPython and the IMU Machine Learning Core Features
- Getting Started With the Nano RP2040 Connect and OpenMV
- Control Built-in RGB LED over Wi-Fi with Nano RP2040 Connect**
- Voice Commands With The Arduino Speech Recognition Engine

Home / Hardware / Nano RP2040 Connect / **Control Built-in RGB LED over Wi-Fi with Nano RP2040 Connect**

i This tutorial refers to a product that has reached its end-of-life status.

Control Built-in RGB LED over Wi-Fi with Nano RP2040 Connect

Learn how to set up your board as a web server, allowing other clients to connect via browser, to control and monitor data.

Author · Karl Söderby · Last revision · 07/17/2024

ON THIS PAGE

- Introduction**
- Goals
- Hardware & Software Needed
- Controlling over Wi-Fi +
- Programming the Board
- Testing It Out +
- Conclusion

Introduction

The Nano RP2040 Connect features a Wi-Fi module and an RGB LED among many other things. In this tutorial we will take a look at how we turn our board into a web server, and control the built-in RGB through a browser.

Note: if you need help setting up your environment to use your Arduino Nano RP2040 board, please refer to [this installation guide](#).

Goals

The goals of this project are:

- Connect to a Wi-Fi network.
- Set up a web server.
- Connect to the server through a browser.
- Control the RGB LED.

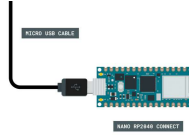
Hardware & Software Needed

Arduino IDE ([online](#) or [offline](#)).

[Wi-FiNINA](#) library installed.

[Arduino Nano RP2040 Connect](#).

Circuit



Plug in the Nano
RP2040 Connect to
your computer.

Controlling over Wi-Fi

There are multiple ways we can access our board over Wi-Fi. In this tutorial, we will turn our board into a web server, that will listen for incoming GET requests.

Simply explained, we will set up our board to connect to a Wi-Fi network, and start hosting a web server on a specific IP address. If we enter this address in the browser of a computer/browser on the same network, we make a request to this server. The server responds with a set of HTML instructions, which can then be viewed in the browser. It works pretty much as the Internet does, but in our example, it will only happen over the local Wi-Fi network.

Controlling an Arduino Through the Browser

Inside the HTML instructions that we print, or send to the browser, we can create **buttons**. These buttons can be modified to add something to the URL. Here is an

```
1 <button class='red' typ
```

The most important is what we put inside `href`, which in this case is `/RH`. Whenever this button is clicked, it will update the URL of the page to `http://<board-ip>/RH`. After this happens, the program will go through a set of conditionals, that checks whether this button has been pressed:

```
1 if (currentLine.endsWit
2   doSomething();
3 }
```

By using this method, we can set up many more buttons that can control different aspects of our Arduino, and is a great building block for building a control interface for your board that can be controlled through a browser, over WiFi.

Programming the Board

We will now get to the programming part of this tutorial.

- 1 First, let's make sure we have the drivers installed. If we are using

anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager....** Here we need to look for the **Arduino Mbed OS Nano Boards** and install it.

- 2 Now, we need to install the libraries needed. If we are using the Cloud Editor, there is no need to install anything. If we are using an offline editor, simply go to **Tools > Manage libraries...**, and search for **WiFiNINA** and install it.
- 3 We can now take a look at some of the core functions of this sketch:

`char ssid[] = ""` - stores network name.

`char pass[] = ""` - stores network password.

`WiFi.begin(ssid, pass)` connects to Wi-Fi with credentials.

`WiFiServer server(80)` - creates a server that listens for incoming connections on the specified port.

`WiFiClient client` - creates a client that can connect to a specified internet IP address.

`server.begin()` - tells the server to begin listening for incoming connections.

`client.connected` - checks

`client.available` - checks for available data.

`client.read` - reads the available data.

`client.print()` - print something to the client (e.g. html code).

`client.stop()` - closes the connection.

The sketch can be found in the snippet below. Upload the sketch to the board.

```
1  #include <SPI.h>
2  #include <WiFiNINA.h>
3
4  //////////please enter
5  char ssid[] = "";
6  char pass[] = "";
7  int keyIndex = 0;
8
9  int status = WL_IDLE
10 WiFiServer server(80
11
12 void setup() {
13   pinMode(LED_R, OUTP
14   pinMode(LED_G, OUTP
15   pinMode(LED_B, OUTP
16   Serial.begin(9600)
17
18   // check for the W
19   if (WiFi.status()
20     Serial.println("
21     // don't continu
22     while (true);
23   }
24
25   String fv = WiFi.f
26   if (fv < WIFI_FIRM
27     Serial.println("
28   }
```

Testing It Out

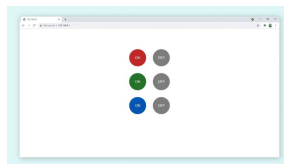
After the code has been

Monitor to initialize the program. As soon as we open it, it will attempt to connect to the Wi-Fi network we entered in the code, and if it is successful, it will print out the board's IP address in the Serial Monitor.



Information in the Serial Monitor.

We now need to copy this IP address and paste it in the browser on a computer/phone on the same network as our board is connect to. We should now see the following page:



Accessing the board from the browser.

We can now interact with the different buttons. The buttons available are used to control the built-in RGB LED on the Nano RP2040 Connect. There's six buttons in total, three to turn ON the different colors, and three to turn them off.

Troubleshoot

If the code is not working, there are some common issues we can troubleshoot:

We have entered the wrong credentials to our Wi-Fi network.

We have not installed the **WiFiNINA** library.

Conclusion

In this tutorial, we have turned our Nano RP2040 Connect into a web server, which different clients can connect to. We have then used the /GET method in order to change the RGB LED on the board, through a set of buttons accessible from the browser.

This method can be quite useful for turning something ON or OFF remotely, and can be an ideal solution for building smart home applications. You can easily replace the control of the RGB LED with something else, such as controlling a motor, relays or other actuators.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

